LLD Learning Series - Day 4

Topics: SOLID Principles Part 2 - LSP Deep Dive, ISP and DIP

Quick Recap of SOLID So Far

Principle	One Line Summary
Single Responsibility (SRP)	A class should have only one reason to change.
Open/Closed (OCP)	Open for extension, closed for modification.
Liskov Substitution (LSP)	Subtypes must be substitutable for their base types without breaking the program.
Interface Segregation (ISP)	Many client-specific interfaces are better than one general-purpose interface.
Dependency Inversion (DIP)	High-level and low-level modules should both depend on abstractions.

SRP, OCP, and LSP were covered conceptually in the previous session. Today we do a deep dive into LSP rules, then cover ISP and DIP in full.

Liskov Substitution Principle - Deep Dive

Definition: Objects of a superclass should be replaceable with objects of a subclass without affecting the correctness of the program.

Inheritance ensures subclasses have the same methods, but not necessarily the same behaviour or contractual guarantees. Without clear rules, a subclass can override a method incorrectly and break client code silently.

The Three Categories of LSP Rules

Category	What it Governs
Signature Rules	Method overrides must preserve the contractual interface of the parent.
Property Rules	Class invariants and lifecycle behaviour must not be violated by the subclass.
Method Rules	Preconditions and postconditions of methods must remain consistent in overrides.

1. Signature Rules

These ensure that method overrides in a subclass are compatible with what clients expect based on the parent's contract.

Method Argument Rule

The overridden method in the subclass must accept the same argument types as the parent, or wider (a broader type up the inheritance chain).

Example: if the parent method takes a String, the child must also take String or a supertype (e.g. Object), never an unrelated type like Integer.

Modern languages like C++ and Java enforce this at compile time.

Return Type Rule

The subclass return type must be the same as the parent's, or narrower (a subtype). This is called covariant return.

Example: parent returns Animal, child can return Dog (a subtype of Animal). Child cannot return Object (broader than Animal).

Exception Rule

The subclass may throw fewer or more specific exceptions than the parent, but never broader or unrelated exceptions the client is not expecting.

Example: parent declares it throws RuntimeError. Child can throw IndexOutOfBoundsException (a subtype), but not OutOfMemoryError if it is not within that hierarchy.

---

2. Property Rules

These ensure the subclass preserves the key properties and expected lifecycle behaviour of the parent class.

Class Invariant

Any invariant (a condition that must always hold true) defined by the parent must not be violated by the subclass.

Example: BankAccount mandates balance >= 0. A CheatAccount subclass that allows a negative balance breaks this invariant and violates LSP.

History Constraint

The subclass must preserve the lifecycle behaviour of the parent. It cannot remove or disable operations that clients expect to always work.

Example: FixedDepositAccount throws an exception on every withdrawal, violating the parent Account's guarantee that withdrawal is always allowed.

---

3. Method Rules - Preconditions and Postconditions

Condition	Definition	Subclass Can	Subclass Cannot
Precondition	Must be true before method executes	Weaken it (accept a broader range)	Strengthen it (require more than parent)

Postcondition	Must be true after method completes	Strengthen it (guarantee more)	Weaken it (guarantee less)

Precondition Example

Parent requires 0 <= x <= 5.

Child can widen to 0 <= x <= 10 (weaker precondition - allowed).

Child cannot restrict to 0 <= x <= 3 (stronger precondition - breaks LSP, clients supplying x = 7 would fail).

Postcondition Example

Parent brake() guarantees: speed decreases after execution.

Child HybridCar can strengthen: speed decreases AND battery charge increases (allowed).

Child cannot weaken: braking has no effect on speed. That violates LSP and could cause dangerous results.

LSP Checklist: Common Violations to Spot

Throwing unexpected or broader exceptions not declared by the parent.

Overriding a method with an empty body or a hard-coded return value.

Returning a type broader than the parent's declared return type.

Accepting a narrower argument type than the parent expects.

Violating a class invariant (e.g. allowing negative balance).

Disabling a method the parent always allowed (history constraint breach).

Interface Segregation Principle (ISP)

Definition: Clients should not be forced to depend on interfaces they do not use. Many small, client-specific interfaces are better than one large, general-purpose interface.

The Problem: Fat Interfaces

A single interface that includes every conceivable method forces some implementers to override methods they do not need. Those methods either throw exceptions or remain unimplemented stubs, hurting maintainability and violating SRP.

Example: Shapes

A fat Shape interface declares both area() and volume(). This forces 2D shapes like Square and Rectangle to implement volume(), which is meaningless for them.

Shape Type	Methods Actually Needed	What Fat Interface Forces
Square, Rectangle (2D)	area() only	area() and volume()
Cube (3D)	area() and volume()	area() and volume()

Solution: Segregate Into Two Interfaces

```
class TwoDShape {
  virtual double area() = 0;
};

class ThreeDShape {
  virtual double area() = 0;
  virtual double volume() = 0;
};

class Square : public TwoDShape { ... };
class Rectangle : public TwoDShape { ... };
class Cube : public ThreeDShape { ... };
```

Each class now implements only the interface relevant to it.

No class is burdened with methods it cannot meaningfully provide.

Code is cleaner and adheres to SRP as well.

Dependency Inversion Principle (DIP)

Definition: High-level modules should not depend on low-level modules. Both should depend on abstractions. Abstractions should not depend on details; details should depend on abstractions.

The Problem: Direct Coupling

A high-level class (e.g. UserService) that directly calls concrete low-level classes (SqlDatabase, MongoDatabase) is tightly coupled to them.

Swapping MongoDB for Cassandra forces changes inside UserService, violating OCP.

Solution: Introduce an Abstraction Layer

```
class Persistence {
  virtual void save(const User& u) = 0;
};

class SqlDatabase : public Persistence { void save(...) override { ... } };
class MongoDatabase : public Persistence { void save(...) override { ... } };

class UserService {
  Persistence* db;
  UserService(Persistence* p) : db(p) { }
  void storeUser(const User& u) { db->save(u); }
};
```

How Dependency Injection Works Here

At runtime, pass either new SqlDatabase() or new MongoDatabase() into the UserService constructor.

UserService calls save() without knowing or caring which database is behind it.

Adding Cassandra support requires only implementing Persistence and injecting the new instance. UserService itself needs zero changes.

Real-Life Analogy

A company CEO (high-level module) never instructs individual developers (low-level modules) directly. Managers (abstraction) relay requirements between them. The CEO depends only on the manager's interface. Developers depend on the manager for directives. Swapping out developers does not affect the CEO's workflow at all.

Final Thoughts: SOLID as Guidelines, Not Laws

SOLID principles are best practices, not rigid rules. Business requirements and performance constraints will sometimes require trade-offs. The goal is to follow these principles as much as possible, and when you must deviate, do so consciously and document your reasoning.

Strict adherence generally leads to more maintainable, scalable, and extensible code. But balance is key over-engineering in pursuit of perfect SOLID compliance can be just as harmful as ignoring the principles entirely.

My Takeaways from Today

1. LSP is not just about inheriting methods. It is about honouring the full contract of the parent: signature, invariants, exceptions, and pre/postconditions.

2. The precondition and postcondition rules are intuitive once seen as a contract: the child cannot demand more from the caller than the parent did, and cannot deliver less.

3. ISP is about keeping interfaces lean. If a class has to write an empty or throw-exception body for a method it does not support, the interface needs to be split.

4. DIP flipped my mental model: instead of the application deciding which database to use, the database is handed to it from outside. This is what dependency injection means at its core.

5. The CEO-Manager-Developer analogy for DIP is the clearest mental model I have encountered for this principle.

6. SOLID principles are not a checklist to tick off. They are design instincts to develop over time. Knowing when to apply them and when to make a pragmatic trade-off is the real skill.